

AIGC检测 · 全文报告单

NO:CNKIAIGC2026SG_20260526151501624

检测时间: 2026-05-26 15:15:01

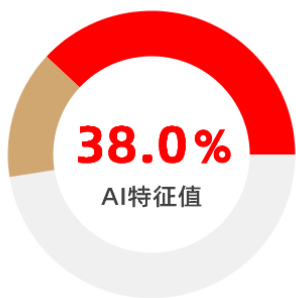
篇名: 基于Gin和WebSocket的IM即时通讯系统

作者: 王浩然

单位:

文件名: 基于Gin和WebSocket的IM即时通讯系统.docx

全文检测结果 知网AIGC检测 <https://cx.cnki.net>



AI特征值: 38.0%

AI特征字符数: 10818

总字符数: 28469

- AI特征显著 (计入AI特征字符数)
- AI特征疑似 (未计入AI特征字符数)
- 未标识部分

AIGC片段分布图

前部20%

AI特征值: 38.2%

AI特征字符数: 2173

中部60%

AI特征值: 37.8%

AI特征字符数: 6460

后部20%

AI特征值: 38.4%

AI特征字符数: 2185



0

28469

■ AI特征显著 ■ AI特征疑似 ■ 未标识部分

分段检测结果

序号	AI特征值	AI特征字符数/章节字数	章节(部分)名称
1	40.2%	4322 / 10764	基于Gin和WebSocket的IM即时通讯系统_第1部分
2	32.7%	3496 / 10686	基于Gin和WebSocket的IM即时通讯系统_第2部分
3	37.1%	3937 / 10602	基于Gin和WebSocket的IM即时通讯系统_第3部分

1.基于Gin和WebSocket的IM即时通讯系统_第1部分

片段指标列表

序号	片段名称	字符数	AI特征		
1	片段1	306	显著		2.9%
2	片段2	126	显著		1.2%
3	片段3	135	显著		1.3%
4	片段4	152	显著		1.4%
5	片段5	761	显著		7.1%
6	片段6	692	疑似		6.4%
7	片段7	707	显著		6.6%
8	片段8	133	显著		1.2%
9	片段9	124	显著		1.2%
10	片段10	163	显著		1.5%
11	片段11	160	显著		1.5%
12	片段12	198	疑似		1.8%
13	片段13	276	显著		2.6%
14	片段14	141	疑似		1.3%
15	片段15	174	显著		1.6%
16	片段16	121	疑似		1.1%
17	片段17	178	显著		1.7%
18	片段18	196	显著		1.8%
19	片段19	173	显著		1.6%
20	片段20	132			

在课题完成过程中，培养严谨求实的科学态度、自主学习与创新精神；通过研究消息加密技术，增强网络空间安全意识与社会责任感，树立正确的网络伦理观念。主要参考资料：

[1] Tanenbaum A S, Wetherall D J. Computer Networks[M]. 5th ed. Boston: Pearson, 2011.

[2] Donovan A A A, Kernighan B W. The Go Programming Language[M]. Boston: Addison-Wesley, 2015.

[3] You E. Vue.js 3 Documentation[EB/OL]. (2024-01-01)[2026-03-01].
<https://vuejs.org/>.

1.2% (87)

完 成 期 限： 2026年5月18日

1.2% (19)

指导教师签名：

1.2% (7)

专业负责人签名：

2026年1月6日

摘 要

即时通讯系统早已不是单纯的聊天工具，而是很多互联网应用里的基础设施。问题在于，常见方案虽然会用 TLS 保护传输链路，可消息一旦进入服务端，往往仍以明文形式保存。

一旦数据库泄露、运维权限失守，真正暴露出去的就是完整聊天内容。本文正是从这个缺口出发，设计并实现了一套基于 Gin 和 WebSocket 的 IM 即时通讯系统，在补齐用户认证、好友关系、实时聊天、历史消息查询这些基础能力的同时，把重点压在端到端消息加密的工程落地上。

1.3% (135)

本系统前端基于 Vue 3 搭配 Vite、TypeScript 和 Pinia，后端用 Go 1.22 配合 Gin、GORM、MySQL，实时通信交给 Gorilla WebSocket 维护的长连接。真正花了心思的是安全实现这一块——选用 RSA-OAEP 配合浏览器原生的 Web Crypto API：密钥对由前端在本地生成，私钥从不离开浏览器，公钥才会传到服务端。

发送一条消息时，前端会拿发送方公钥和接收方公钥各加密一次，得到两份独立密文；服务端拿到的只是密文，做的事情也只剩校验、入库和转发。这样一来，聊天历史在两端都能读到，但明文从未越过浏览器这道边界。功能测试共设置 12 项用例，覆盖注册登录、好友管理、实时聊天、历史消息查询和消息加密，结果全部通过。

1.4% (152)

抓包以及数据库检查也没有发现明文消息残留。换句话说，这套方案至少把端到端加密最关键的一步走通了：服务端看不到消息内容，通信双方还能在本地回看历史。与此同时，文中也保留了几个尚未解决的问题，包括公钥真实性校验、前向保密、会话密钥协商和多设备同步，它们将决定系统后续能否继续往工业级 E2EE 靠近。

关键词：即时通讯；前后端分离；WebSocket；端到端加密；RSA-OAEP

7.1% (39)

ABSTRACT

7.1% (8)

Instant messaging (IM) has become one of the most important application forms in the Internet era, and is widely used in social communication, office collaboration and online cooperation scenarios. As users pay more and more attention to the privacy of their messages, how to provide effective content encryption protection in a general-purpose IM system has become an important research and engineering topic. This paper designs and implements an instant messaging system based on Gin and WebSocket, and focuses on the engineering implementation of end-to-end message encryption while retaining basic functions

7.1% (699)

such as authentication, friend management, real-time chat and historical message query.

The front-end of the system is built on Vue 3, Vite, TypeScript and Pinia, while the back-end is implemented with Go 1.22, Gin, GORM and MySQL. Real-time message delivery is supported by Gorilla WebSocket. In the security design, RSA-OAEP and the browser-native Web Crypto API are adopted. Key pairs are generated locally in the browser, private keys are kept on the client side only, and public keys are uploaded to the server. For each outgoing message, the plaintext is encrypted twice with the sender's public key and the receiver's public key respectively, so that the server stores and forwards ciphertext only while both communication parties can decrypt their own copies locally.

Experimental and functional verification results show that all 12 core test cases pass successfully, and no plaintext message is exposed in the server-side storage or network payloads. The proposed scheme exhibits the basic characteristics of end-to-end encryption and is suitable for an undergraduate engineering project. At the same time, the limitations in public key authenticity verification, forward secrecy, session key negotiation and multi-device synchronization are explicitly discussed, providing a clear direction for future evolution toward a full industrial-grade E2EE system.

KEY WORDS: Instant Messaging; Front-end and Back-end Separation; WebSocket; End-to-End Encryption; RSA-OAEP

6.4% (687)

6.6% (590)

6.6% (107)

目 录

摘 要I

ABSTRACTII

1 绪论1

1.1 研究背景与意义1

1.2 国内外研究现状2

1.2.1 国外研究现状2

1.2.2 国内研究现状3

1.3 论文主要工作3

1.4 论文组织结构4

2 相关技术介绍5

2.1 前后端分离架构5

2.2 Vue 3 与前端技术栈5

2.3 Go 语言与 Gin 框架6

2.4 WebSocket 协议6

2.5 JWT 身份认证7

2.6 RSA-OAEP 公钥加密与 Web Crypto API7

2.7 MySQL 数据库与 GORM8

3 系统需求分析与总体设计9

3.1 需求分析9

1.2% (34)

1.2% (20)

1.2% (14)

1.2% (9)

3.1.1 功能性需求9	1.2% (12)
3.1.2 非功能性需求10	1.2% (14)
3.1.3 用例分析10	
3.2 系统总体架构11	
3.3 系统功能模块设计12	
3.4 数据库设计13	
4 关键技术实现15	
4.1 用户认证与 JWT 实现15	
4.2 好友关系处理16	
4.3 基于 WebSocket 的实时聊天实现17	
4.3.1 WebSocket 连接建立与鉴权17	
4.3.2 连接管理与消息广播18	
4.3.3 消息收发完整时序19	
4.4 端到端消息加密机制20	
4.4.1 设计目标与总体思路20	
4.4.2 RSA-OAEP 密钥对生成与管理21	1.2% (25)
4.4.3 公钥上传与好友公钥获取22	1.2% (19)
4.4.4 双重密文加密与服务端密文转发22	1.2% (22)
4.4.5 接收端解密与历史消息展示23	1.2% (20)
4.4.6 安全性分析24	1.2% (13)
5 系统实现与测试25	
5.1 开发与运行环境25	
5.2 用户认证模块实现25	
5.3 个人资料模块实现26	
5.4 好友管理模块实现27	
5.5 聊天模块实现28	
5.6 系统功能测试29	
5.7 系统性能与安全性测试30	
5.8 与同类系统的对比31	
结束语33	
致 谢34	
参考文献35	
1 绪论	
1.1 研究背景与意义	
聊天工具的历史从 ICQ、QQ 一路走到微信，几乎与中国互联网用户一起长大。CNNIC 2024 年的数据里有一个不太引人注意的细节：国内即时通讯用户已突破 10 亿——它的覆盖面，差不多就是“上网的中国人”这个集合本身。聊天框里流过的早已不只是闲聊，私人对话、企业的合同洽谈、远程协作中的商业细节都挤在同一根管道里。	1.5% (159)
安全性，从过去的“锦上添花”一步步被推成了硬指标。另一个绕不开的趋势是 Web 工程的形态变化。前端被 Vue、React 这类单页框架接管，后端往往只留 HTTP/HTTPS 接口和 WebSocket 端点对外。落到 IM 场景里，这种分工反而非常顺手：HTTP 处理注册	

、好友、历史消息这种请求一响应业务恰到好处，而真正吃延迟的实时消息，更适合丢给 WebSocket——全双工、长连接、低延迟，这几个性质几乎就是为聊天准备的。近几年 IM 数据泄露的新闻并不少。2018 年某社交平台因服务器配置失误，数百万条聊天记录被外部爬取；2021 年一家公司内部 IM 的运维账号被钓鱼，历史消息整库被搬走。

两起事故有一个共同点很值得说一下——攻击者根本没碰加密这一层。只要拿到服务端权限，他们要的就是明文。这件事的含义其实很明确：传输层加密远远不够，安全防线必须再往客户端那一端推。现在主流 IM 普遍是“TLS 加密通道 + 服务端明文存储”这套组合：传输做得很扎实，可消息一旦写入数据库，就以明文形式静静躺在那里。

1.5% (160)

脱库、运维误操作、内部权限滥用——任何一个环节出问题，整张聊天记录表都会一锅端。WhatsApp、Signal 这些产品近年陆续把端到端加密（E2EE）拉进协议层，原因恰恰就在这里：让服务端“看不到”消息，不再是一个加分项，而是底线。本文的工作就是顺着这个痛点往下做：先搭一套前后端分离的 IM 系统，把注册登录、好友、聊天、历史消息这些常规功能跑起来，再把端到端加密真正塞进业务链路里。

1.8% (198)

算法没有另起炉灶，而是选了公开、成熟、经同行评议较多的 RSA-OAEP。这样做首先是工程问题——浏览器、WebSocket 和数据库这条链路必须完整接起来；接着才谈技术验证，也就是 Web Crypto API 放进通用浏览器之后，究竟能不能稳定支撑 E2EE。说到底，落点还是安全：让服务端摸不到明文，把最常见的明文泄露路径尽量堵住。

1.2 国内外研究现状

1.2.1 国外研究现状

国外对即时通讯的研究启动得很早。1996 年 ICQ 横空出世，把“在线状态 + 即时聊天”的概念带给大众；之后 AOL、Yahoo、MSN 的同类产品相继出现，直接推动了 XMPP、IRC 等协议的标准化。其中 IETF 主导的 XMPP 一度被视为开放协议的代表，Google Talk、Jabber 都以它为底座。近十年的风向明显转向安全。Open Whisper Systems 提出的 Signal 协议是其中绕不开的代表作——它把 X3DH 密钥协商和 Double Ratchet 双棘轮算法揉在一起，实现了前向保密与未来保密这两项硬指标，今天被 WhatsApp、Signal、Messenger 的“隐私对话”广泛使用。Bellare 等学者随后从密码学角度给出了形式化证明，把工程上行得通的协议从理论上又夯实了一遍。

另一条值得关注的支线是 Matrix.org——在去中心化的基础上做端到端加密，提供了与 Signal 完全不同的拓扑思路。浏览器侧的端到端加密也不是近几年才突然冒出来的想法。2017 年 Web Crypto API 成为 W3C 推荐标准之后，JavaScript 终于可以直接调用浏览器原生密码学原语。随后 ProtonMail、Tutanota 这类 Web 安全邮件服务陆续证明了一件事：把加密放在浏览器本地、把服务端压缩成密文存储与转发节点，这条路在工程上是能走通的。后来 Web 端 IM 想做端到端加密，很多经验其实就来自这里。

2.6% (276)

1.2.2 国内研究现状

国内的故事得从 1999 年的 OICQ 讲起。它后来改名 QQ，又一路演化成今天微信这种把通讯、社交、支付揉在一起的超级应用。腾讯、阿里、字节这些公司围绕海量长连接、分布式存储、多端同步、群消息扩散踩过的坑相当多，沉淀下来的方案——基于消息队列的离线投递、基于一致性哈希的连接分配、Read/Write Diffusion 群推送策略——在工业界已经相当成熟。

学术层面，国内研究大致分布在几个方向：WebSocket 在高并发场景下的性能调优；用 Go、Erlang 这类天生擅长并发的语言搭消息推送架构；还有 IM 中的消息加密与隐私保护——从 RSA、AES 到 ECC，相关论文讨论了不同算法在 IM 场景下的可行性与性能边界。

1.3% (137)

1.6% (174)

但说实话，问题摆在那里：工程实现已经很成熟，端到端加密却还停留在头部厂商内部，本科课程项目里几乎是空白——这正好是本课题想补的位置。另外，OpenIM、go-chat 这一批基于 Go / Rust 的轻量开源 IM 项目，近两年在校园项目和企业内部协作工具里出现得越来越多。它们给本课题提供的，主要不是现成代码，而是分层架构上的直接参照。

本系统沿用了这类项目里比较成熟的工程思路，再把 Web Crypto API 和 RSA-OAEP 放进浏览器侧，尝试做出一套既能演示、也能验证的端到端加密方案。若把尺度放在课程项目里看，这已经算是在“轻量级 E2EE”这条路上往前走了一步。

1.1% (121)

1.3 论文主要工作

本文围绕“前后端分离 IM + 端到端加密”展开，具体做了以下几件事。需求和设计这一部分，先梳理了即时通讯的发展脉络与典型架构，再从功能性与非功能性两条线整理需求，用 UML 用例图把用户与系统之间的交互画清楚，随后给出总体架构、模块划分与数据库 E-R 设计。

后端实现使用 Go + Gin + GORM + MySQL，完成用户认证、个人资料、好友关系、消息存储与转发等业务模块；实时通信部分则交给 Gorilla WebSocket。前端使用 Vue 3 + Vite + TypeScript，配合 Pinia 做状态管理、Axios 统一封装 HTTP，覆盖注册、登录、资料、好友、聊天五个业务页面。

1.7% (174)

真正花心思的部分还是加密。RSA-OAEP 密钥对在浏览器侧生成并保存在本地，服务端不碰私钥，只接收、存储和转发密文。同一条消息会同时生成发送方副本和接收方副本，这样双方都能在自己的设备上回看历史。最后再通过功能测试、接口测试和几组安全性观察，把这套实现从不同角度跑了一遍。

1.4 论文组织结构

全文共分 5 章。第 1 章交代选题背景、国内外现状与本文要做的工作；第 2 章把后续要用到的技术过一遍——前后端分离、Vue 3、Go 与 Gin、WebSocket、JWT、RSA-OAEP、MySQL；第 3 章从需求分析切入，给出总体设计与数据库设计；第 4 章是技术实现的主体，其中 WebSocket 实时聊天与端到端加密是两大重点；第 5 章给出实现细节、运行截图与测试结果。结束语、致谢与参考文献放在正文之后。

2 相关技术介绍

2.1 前后端分离架构

前后端分离的本质，是把视图和业务这两件原本耦合在一起的事拆开来：前端工程跑在浏览器里，专心做渲染、交互和状态；后端工程跑在服务器上，只负责业务、数据持久化和接口暴露。两边通过 RESTful JSON 接口和 WebSocket 长连接对话。相比传统服务端渲染，这种结构对团队协作和水平扩展更友好——前端、后端可以各走各的迭代节奏，同一套接口既能撑 Web，也能复用到移动端或桌面端。

1.8% (192)

本系统前端用 Vue 3，后端用 Go + Gin，走的就是这条主流路线。

2.2 Vue 3 与前端技术栈

Vue 3 是一套响应式驱动的渐进式前端框架。相较于 Vue 2，它引入了 Composition API、基于 Proxy 的新响应式系统，以及对 TypeScript 更完整的支持。本系统以前端 Vue 3 为核心，配套使用 Vite、TypeScript、Pinia、Vue Router 和 Axios。这里每个工具的职责都比较明确：Vite 负责构建，TypeScript 帮助在编译期拦住低级错误，Pinia 管理状态，Vue Router 负责路由与登录拦截，Axios 统一处理 HTTP 请求和 JWT 注入。

2.3 Go 语言与 Gin 框架

后端选 Go，首先还是看中它处理并发连接时足够顺手。IM 单机要维护大量 WebSocket 长连接，如果每条连接都压在线程模型上，开销会很快上来；Go 的 goroutine 正好适合这种场景，每个连接分到一个轻量执行单元，调度交给运行时接管。除此之外

1.6% (169)

，Go 的语法和部署方式也很省心：代码风格统一，编译后就是单一二进制，交付成本不高。

Gin 是 Go 生态里使用最多的 Web 框架。它在 net/http 之上做了 Radix Tree 路由优化，并提供一套优雅的中间件机制——本系统正是借此把 CORS、JWT 鉴权、日志统一塞到中间件层，业务 Handler 里几乎看不到模板代码。WebSocket 升级请求也走同一套路由系统，HTTP 与 WS 在工程上不需要两套基础设施。Go 还有一处常被低估的优点，在于它的工程文化很统一。

gofmt、go vet、go test 这几件工具放在一起，基本把代码格式、静态检查和测试底线都钉住了。这样一来，协作时很少会把时间耗在无谓的风格争论上。对毕业设计这种本来人手和时间都有限的项目来说，这种统一反而很实用。

2.4 WebSocket 协议

在 WebSocket 出现之前，浏览器想做“准实时”通信，只能靠 Comet 长轮询或者 HTTP 流——前者频繁建连销毁，后者难以穿透代理。WebSocket 用一次握手把这个老问题彻底解决。WebSocket 由 RFC 6455 定义，是一种基于 TCP 的全双工协议。

与 HTTP 的“一次请求——一次响应”不同，WebSocket 在握手完成后会保持长连接，双向都能随时推送消息，正好契合 IM、在线协作、推送通知这类场景。它的握手阶段复用 HTTP，通过“Upgrade: websocket”头把 HTTP 连接转成 WebSocket，之后的字节流就不再受 HTTP 模型约束了。

本系统用 WebSocket 来承载实时下行：客户端通过 GET /ws?token=XXX 携带 JWT 完成握手，服务端校验后将连接升级为 WS 并注册到统一的 Hub。当 Hub 收到某条新消息时，按接收方 ID 查找活跃连接并下发密文。

后端用的 Gorilla WebSocket 是 Go 生态事实上的标准实现，比标准库稳定不少。

2.5 JWT 身份认证

JWT (JSON Web Token) 由 RFC 7519 定义，是一种紧凑、自包含的令牌格式，在前后端分离系统里通常用来替代 Session。

它由 Header、Payload、Signature 三段组成，中间用点号连接并做 Base64URL 编码。Payload 里可以塞用户 ID、过期时间等业务字段，Signature 则保证整串令牌没被篡改。JWT 比 Session 更有吸引力的一点在于：服务端不需要再维护会话状态。这对横向扩展尤其友好；同一份令牌天然可以在 Web 与移动端等多客户端之间通用；如果把业务上下文写进 Payload，连一次数据库查询都能省下。

本系统在登录成功后由后端签发 JWT，前端把它缓存在 localStorage，Axios 拦截器自动加到请求头上，后端中间件统一做校验。WebSocket 握手时则改走 URL 查询参数，鉴权口径保持一致。

2.6 RSA-OAEP 公钥加密与 Web Crypto API

RSA 是现在用得最广的非对称加密算法，1977 年由 Rivest、Shamir、Adleman 三位学者提出，安全性建立在大整数分解困难这一假设之上。它真正巧妙的地方在那对密钥：公钥负责加密，私钥负责解密，两者数学上彼此相关，却推不出来。

这一性质把困扰对称加密多年的密钥分发问题一笔勾销——公钥可以在任何信道上公开传播，私钥则永远留在本地。攻击者哪怕拿到了公钥，也对密文无可奈何。不过裸用 RSA 是危险的。PKCS #1 v1.5 填充在选择密文攻击下早已被攻破，因此现代实现必须配合更安全的填充模式——OAEP。

OAEP 引入随机数和 MGF 掩码生成函数，让相同明文每次加密出的密文都不同，从根本上挫败选择密文攻击。本系统选用的就是 RSA-OAEP-SHA256：SHA-256 同时充当 MGF 的内部哈希和 label 的哈希，安全强度符合当前业界对 RSA 的推荐配置。

Web Crypto API 是 W3C 规范里的浏览器原生加密接口，提供了密钥生成、对称/非对

1.2% (131)

1.4% (143)

1.2% (122)

1.3% (64)

1.3% (72)

1.3% (141)

1.3% (143)

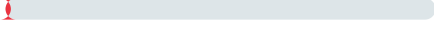
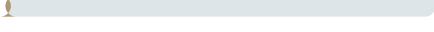
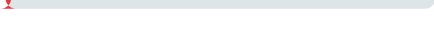
1.2% (134)

称加解密、签名、哈希这些基础原语。最关键的一点是——所有运算都在浏览器进程内部完成，不依赖任何第三方 JS 加密库。这意味着两件事：性能比纯 JS 实现高出至少一个数量级，安全性也更可靠，因为算法实现来自浏览器厂商，而不是某个随手安装的 npm 包。本系统调用的就是 window.crypto.subtle，由前端直接完成 RSA-2048 密钥对的生成、导入导出与消息加解密。

2.基于Gin和WebSocket的IM即时通讯系统_第2部分

AI特征值：32.7% AI特征字符数/章节(部分)字符数：3496/10686

片段指标列表

序号	片段名称	字符数	AI特征		
101	片段1	175	显著		1.6%
102	片段2	232	显著		2.2%
103	片段3	186	显著		1.8%
104	片段4	126	显著		1.2%
105	片段5	128	疑似		1.2%
106	片段6	169	疑似		1.6%
107	片段7	287	疑似		2.7%
108	片段8	140	显著		1.3%
109	片段9	122	疑似		1.1%
110	片段10	195	疑似		1.8%
111	片段11	127	显著		1.2%
112	片段12	166	疑似		1.6%
113	片段13	186	显著		1.8%
114	片段14	189	显著		1.8%
115	片段15	120	疑似		1.1%
116	片段16	152	显著		1.4%

117	片段17	178	疑似		1.7%
118	片段18	173	显著		1.6%
119	片段19	163	疑似		1.5%
120	片段20	190	显著		1.8%
121	片段21	210	显著		2.0%
122	片段22	158	疑似		1.5%
123	片段23	128	显著		1.2%
124	片段24	135	疑似		1.3%
125	片段25	154	显著		1.4%
126	片段26	256	显著		2.4%
127	片段27	196	显著		1.8%
128	片段28	160	显著		1.5%
129	片段29	160	疑似		1.5%
130	片段30	123	显著		1.2%
131	片段31	136	显著		1.3%
132	片段32	257	显著		2.4%
133	片段33	146	疑似		1.4%
134	片段34	140	疑似		1.3%

原文内容

2.7 MySQL 数据库与 GORM

MySQL 几乎是开源关系型数据库的代名词，生态成熟、运维成本低、性能足够撑起绝大多数中小型 Web 应用。本系统采用 MySQL 8.x，设计了 users、friend_requests、friends、messages 四张核心数据表，覆盖账号、好友申请、好友关系与消息四个领域。

GORM 则是 Go 生态里最流行的 ORM，提供链式 API、自动迁移、关联查询和事务管理

，把大部分原生 SQL 工作收敛到结构体声明里。本系统在 Repository 层使用 GORM 处理数据访问，通过 struct tag 显式声明字段映射，在开发效率与 SQL 可控性之间留了一条退路——复杂查询需要时也可以直接写 Raw SQL。

3 系统需求分析与总体设计

3.1 需求分析

写代码之前先要把“系统要做什么、做到什么程度”想清楚。下面分别从功能、非功能和用例三个方向展开。需要交代一句：本课题没有外部客户，需求是从典型 IM 系统的业务闭环倒推出来的——以“用户能完成一次完整聊天”作为最高目标，再逐步拆到每一个具体接口和页面。

这种自顶向下的拆法在中小型项目里最实用，能避免把精力浪费在用不上的功能上。

2.2% (52)

3.1.1 功能性需求

结合典型 IM 的业务闭环和本课题的加密目标，功能性需求大致落在六个模块上。

用户认证：注册和登录都用“用户名 + 密码”走，注册时校验用户名唯一，密码必须哈希后再落库；登录成功后由服务端签发 JWT，前端后续请求把令牌带上即可。个人资料：登录用户可以查看与修改昵称、头像、性别、个性签名等基本信息，并能看到自己当前的公钥是否已经就绪。

2.2% (175)

好友关系：通过用户名发起好友申请，对方可以选择同意或拒绝；一旦同意，双方互相出现在好友列表中。实时聊天：好友之间走 WebSocket 长连接收发消息，系统需要妥善处理双方都在线、单方在线、双方都不在线这几种情形。消息加密：用户第一次进入聊天功能时，前端自动生成 RSA 密钥对——私钥留在本地，公钥推到服务端；发送一条消息时前端做双重加密后上传，服务端只接收密文，接收方在浏览器里用私钥解开。

历史消息：用户可以回看与某位好友的聊天记录，服务端给出密文，前端本地解密后展示。

1.7% (56)

3.1.2 非功能性需求

非功能性需求里，本系统给自己设了几条线，并不是均匀分配的，重要程度有先后。排在最前面的当然是安全：密码必须经 bcrypt 这类单向哈希再保存，所有受保护接口走 JWT 鉴权，消息明文不能在服务端落地，私钥也绝不能上传——这几条没有商量余地。

1.7% (121)

其次是实时性：在双方都在线、网络条件正常的局域网/本机环境下，从点击发送到对端解密渲染，端到端延迟控制在 100 ms 以内。可扩展性方面，前后端分离配合分层架构，业务、数据访问、连接管理之间用清晰的接口隔开，为后续群聊、离线消息、已读回执这些功能留出接口。余下两条相对琐碎一些：前端交互要贴近常见社交软件的习惯——连接状态、加密状态都应当能被用户看见；代码层面命名规范、层次分明，关键接口要有单元测试与集成测试覆盖。

3.1.3 用例分析

本系统里只有一类典型角色——“已注册的普通用户”，他与系统之间的交互如图 3-1 所示。

注册、登录、查看与修改资料、上传公钥；发送和处理好友申请、查看好友列表；建立 WebSocket 连接、收发加密消息、查看历史消息——一张图把所有用例串起来了。后面模块怎么切、接口怎么排，几乎都是从这张图里读出来的。

1.2% (125)

图 3-1 系统用户用例图

其中“上传公钥”这个用例值得单独说一句——它对用户是完全透明的，首次进入聊天功能时由前端自动触发，可它又是整套加密体系能否工作的前提。在用例图里把它显式画出来，是想提醒读者：哪怕是看起来不起眼的“自动行为”，在系统设计阶段也应当被当作独立用例对待。

1.2% (124)

3.2 系统总体架构

总体架构沿用最典型的“前端 + 后端 + 数据库”三段式，如图 3-2 所示。前端跑在浏

浏览器里，业务请求走 HTTP、实时消息走 WebSocket，加解密由 Web Crypto API 在本地完成。后端是这套系统真正的“主战场”。从外到内分为六层——路由层把 HTTP 请求和 WebSocket 升级请求分发到 Handler；中间件层统一处理 CORS、JWT 鉴权与日志；Handler 层做参数绑定与响应组装；Service 层承载真正的业务逻辑；Repository 层借助 GORM 与数据库打交道；WebSocket Hub 层独立出来，专门维护在线连接并按用户 ID 推送下行消息。各层职责清晰，调用关系单向向下，避免循环依赖。数据库选 MySQL 8.x，保存用户、好友申请、好友关系与消息四类数据。

有一点要强调：messages 表里存的全部是密文加上算法标识，没有任何明文。为什么不引入 Redis 或消息队列？本系统的目标用户规模在课程项目量级，WebSocket 单机维护几千条连接、MySQL 处理读写完全够用。过早引入中间件只会增加部署成本和论文外的额外依赖，反而模糊本课程真正想讲的事——“把端到端加密做进 IM”。

换个说法，架构上的克制本身就是一种设计选择。图 3-2 系统总体架构图

3.3 系统功能模块设计

按高内聚、低耦合切分，系统划成 5 个核心模块，前后端的对应关系如图 3-3 所示。图 3-3 前后端模块结构图

用户认证模块负责注册、登录、令牌签发、令牌校验与登出，后端落在 AuthHandler/AuthService，前端落在 LoginView、RegisterView 以及 auth store。个人资料模块处理昵称、头像、性别、签名等基础信息的查改，也包括公钥的上传——后端是 UserHandler/UserService，前端则是 ProfileView。好友关系模块覆盖申请的发送、接收、同意、拒绝以及好友列表查询，后端 FriendHandler/FriendService，前端 FriendRequestsView 和 FriendsView。

实时聊天模块是 HTTP 与 WebSocket 协作的舞台：MessageHandler 负责密文入库，MessageService 校验业务规则，WebSocketHandler 完成实时推送。消息加密相关逻辑被单独抽到一个文件里——前端 utils/e2ee.ts。

它把 RSA-OAEP 的密钥生成、导入导出、加解密、本地持久化、历史消息密文选择这些零碎逻辑全部封装起来，上层视图只需要“获取密钥、加密、解密”这三个动作就能用上端到端加密能力。这样做的好处是加密细节不会渗透进业务代码，将来想换算法也更省事。

3.4 数据库设计

从需求里抽出的实体落到 4 张核心表：users、friend_requests、friends、messages，E-R 关系如图 3-4 所示。users 与后三张表都是一对多——一个用户可以发出多条好友申请、拥有多个好友、收发多条消息。图 3-4 数据库 E-R 图

users 表除常规字段外，额外增加 public_key 与 public_key_algorithm 两列，保存用户在浏览器侧生成的公钥及算法标识。其他用户向该用户发消息时，正是从这两列拿到加密用的公钥。friend_requests 表记录申请本身——from_user_id、to_user_id 加一个 status 标识申请处于“待处理/同意/拒绝”哪一种状态。

一旦 status 被改为“同意”，业务层会在 friends 表里插入双向记录，保证 A 查好友能看到 B、B 查好友也能看到 A。friends 表用 (user_id, friend_id) 联合唯一索引避免重复落库。索引方面专门做过取舍。

users 给 username 加唯一索引，登录与防重复注册一并解决；friend_requests 给 (to_user_id, status) 加复合索引，专门为“我收到的待处理申请”这条高频查询服务；friends 给 user_id 建普通索引，加速好友列表；messages 建 (sender_id, receiver_id, created_at) 三列复合索引，把历史消息的范围扫描压在 O(log n) 量级。

1.6% (169)

2.7% (283)

1.3% (140)

1.1% (122)

1.8% (191)

1.2% (127)

索引不是越多越好，写多读少的字段就没必要建——这一点在落地时也做了权衡。
messages 表是整个安全设计的落地点。除 sender_id、receiver_id、created_at 外，专门设了 sender_ciphertext / sender_algorithm 与 receiver_ciphertext / receiver_algorithm 两组字段。sender_ciphertext 用发送方公钥加密，receiver_ciphertext 用接收方公钥加密——同一段明文加密两次，收发双方各拿一份能用自己私钥解开的副本，历史消息双向可读的同时，服务端依然看不见明文。 4 关键技术实现

本章按实现链路往下展开。4.1 节先交代用户认证与 JWT，4.2 节说明好友关系如何落库和校验，4.3 节进入 WebSocket 实时聊天，4.4 节再把重点放到端到端加密上。真正能拉开系统差异的，主要还是后两节。 要先说明一下：4.1 与 4.2 节走的是相对常规的实现路径，更多是把系统的基础设施搭起来；真正能体现本文价值的，是 4.3 节里 WebSocket 的连接管理，以及 4.4 节那套端到端加密的设计取舍。 4.1 用户认证与 JWT 实现

用户认证算是整个系统的入口——后续所有业务都建立在“这条请求来自哪个用户”这件事上。后端把注册和登录拆成 AuthService 里的两个独立方法。密码不能明文存储，这是底线；本系统选 bcrypt 是因为它内置盐值，并且允许调节工作因子，离线穷举的成本会随着硬件升级一起水涨船高——比裸的 MD5 或 SHA1 稳健得多。

1.6% (162)

登录通过后，服务端用 HMAC-SHA256 签发一个 JWT。Payload 里塞着用户 ID 和过期时间戳，签名密钥只保留在服务端配置里，绝不下发。前端拿到 Token 之后存进 localStorage，HTTP 请求里走 “Authorization: Bearer <token>” 头，WebSocket 握手则改走 URL 查询参数——两条路径同一份令牌，验签逻辑完全复用。

受保护接口的鉴权交给 AuthMiddleware 统一处理。中间件取出请求头里的 JWT，用同样的密钥验签：过期、伪造或缺失，全部直接 401 拒绝；验签通过则把用户 ID 放进 gin.Context，业务 Handler 只管 c.Get(“userId”) 就行。这种“一道关卡守住全部接口”的写法，让 Handler 里几乎看不到任何鉴权相关的样板代码。

1.7% (182)

bcrypt 工作因子，本系统取的是默认值 10，一次哈希大约 60-80 ms。这个量级对登录接口完全可接受，但对暴力破解者来说意味着每秒只能尝试十几次，相比 MD5 那种数十亿次/秒的速度，几乎是压倒性差距。万一 GPU 算力又上一个台阶，只要把工作因子改成 11 甚至 12 重新加密一遍，系统的防护强度就能跟上来。 关键中间件代码如下：

1.8% (39)

```
func AuthMiddleware() gin.HandlerFunc {
```

1.8% (10)

4.2 好友关系处理

好友关系由两张表协同支撑——friend_requests 记录申请，friends 记录已建立的关系。业务流程不复杂，但有几个细节需要拿捏。 用户 A 给 B 发申请时，后端要先做三道前置校验：B 是否存在、是否已经是好友、是否还有未处理的旧申请。

1.8% (128)

任何一道没过，请求直接被驳回；都过了，才在 friend_requests 表里写一条 status=待处理 的记录。这一步把“重复申请”挡在数据库之外，省得后面再靠唯一索引兜底。 B 在申请页处理时——同意或拒绝——会触发不同分支。

1.1% (120)

同意路径走数据库事务，把 friend_requests 的 status 改为“已同意”的同时，向 friends 表插入 (A, B) 和 (B, A) 两条记录。为什么是两条？因为这样 A 查好友能看到 B、B 查好友能看到 A，查询路径上完全对称，业务代码不需要做“双向 OR”判断，逻辑更清爽。

1.4% (152)

拒绝路径则只更新状态，friends 表保持不动。 聊天与历史消息接口在执行任何业务之前，都会先调用 FriendService.IsFriend(senderId, receiverId)，把“非好友越

权发消息”这条攻击面直接堵死。

把校验放在 Service 层而不是 Handler 层，是为了让 HTTP 和 WebSocket 两条入口都强制走同一道闸门。

4.3 基于 WebSocket 的实时聊天实现

实时聊天是这套系统里最见工程功夫的一部分。轮询和长轮询并非完全不能用，但放到 IM 场景里，低延迟、持续在线、服务端主动下行这几件事会很快把它们的短板暴露出来。

WebSocket 更贴近这里真正需要的通信方式：客户端不必反复发起请求，服务端也能主动把新消息推到目标连接。下面不按抽象原理讲，而是顺着实现路径往下拆：先看连接怎么建，再看连接怎么管，之后处理异常，最后把整条消息链路串起来。

4.3.1

WebSocket 连接建立与鉴权

WebSocket 在 RFC 6455 里复用 HTTP 完成握手——客户端发一个带 “Upgrade: websocket” 头的 GET，服务端校验后返回 101 完成协议切换。

握手本质上还是一次 HTTP 请求，这就给鉴权留下了天然的入口：本系统选择把 JWT 通过 URL 查询参数 token=<jwt> 传给服务端，在握手阶段就完成验证。另一种常见做法是“先放行连接、再让客户端首帧发 token”。但这样做有两个问题：未鉴权连接会提前占用服务端资源；逻辑上要在 WS 层再绕一层“伪鉴权握手”，反而更复杂。

把校验前移到握手阶段更直接，代价是 token 会出现在 URL 里——反向代理和访问日志可能都会记下来，所以 token 的过期时间要短，敏感字段更不要往 Payload 里塞。

WebSocket 升级处理的关键代码如下：

```
func (h *WSHandler) Serve(c *gin.Context) {
```

有一处细节容易被忽略：WebSocket 握手虽然最终会升级为长连接，但它的入口仍然是一条标准 HTTP 请求，因此同样要穿过 Gin 的中间件链路。跨域响应头、统一日志这些横切逻辑可以直接复用。不过鉴权策略不能照搬普通接口——HTTP 接口的 JWT 一般放在 Authorization Header 里，而 WebSocket 握手更适合从查询参数里读 token。

如果硬要用同一套中间件，势必会出现来源冲突和责任边界混乱。本系统因此把 /ws 单独划成一组路由，JWT 校验放在 Handler 内部自己做。说到底，这种做法是把“通信协议升级”和“业务身份确认”绑在同一个入口完成，避免未鉴权的连接溜进连接管理层。

4.3.2 连接管理与消息广播

WebSocket Hub 是整个实时聊天模块里的“调度中心”。它的内部结构其实很朴素：一张并发安全的 map，key 是用户 ID，value 是该用户当前持有的所有连接列表。新连接进来追加到对应用户的列表里，断开就从列表里摘掉，需要推送时取出活跃连接逐一写入。允许一个用户同时挂多条连接，多标签页或多设备在线的场景就被自然支持了——不必单独处理。

连接管理上真正体现工程差异的地方在并发控制和广播策略。

本系统采用“短临界区 + 锁外写”的方式管理广播：先在读写锁的保护下复制目标用户的连接列表，然后到锁外再逐一执行 conn.WriteMessage。为什么要这么写？因为网络 I/O 的耗时远高于内存读写——如果在持锁状态下直接写 Socket，单个慢客户端就有可能把整个 Hub 的广播路径拖死，进而把锁竞争放大。

换个说法，这种设计本质上是在“一致性开销”和“吞吐能力”之间做选择：允许连接列表在极短时间里和真实状态有些许偏差，换来广播路径整体的性能稳定。对写入失败的连接，系统会主动摘除并关闭底层 TCP，防止无效连接长期滞留。

为什么要允许一个用户挂多条连接？

这是 IM 系统里一个常见但容易被忽略的细节。现实里用户完全可能同时挂在公司电脑、家里的笔记本和手机网页上——如果 Hub 强行只保留最新一条，前面那些客户端就会

1.7% (93)

1.7% (80)

1.6% (173)

1.5% (116)

1.5% (43)

1.8% (186)

2.0% (206)

1.5% (158)

1.2% (128)

出现“假装在线但收不到消息”的情况。本系统选择追加而不是覆盖，把所有活跃连接都纳入推送范围；代价是同一条消息要给同一用户多写几次，但和“悄无声息的消息丢失”相比，这点开销可以接受。

当 Hub 找不到目标用户的活跃连接时，系统会把这位用户当作离线处理。这里的关键不在“是否即时送达”，而在“消息是否最终可达”。本系统刻意把“消息存储”和“消息实时投递”拆成两个层次：HTTP 写库负责可靠持久化，WebSocket 负责在线状态下的低延迟下发。

所以即便接收方离线、推送失败或者网络中断，消息都还在 messages 表里完整保留着；等用户重新进入会话页面，前端通过 GET /api/messages?friendId=xxx 拉回历史密文，在本地解密后再展示。这种做法事实上放弃了“绝对实时必达”的强承诺，换来更简单、更可验证、不易丢消息的实现路径。

对一份毕业设计来说，这是合理且必要的取舍。

4.3.3 异常处理与重连机制

现实的网络远没有局域网那么稳定——网络切换、Wi-Fi 抖动、服务端重启都可能让 WS 连接突然失效。前端的 utils/websocket.ts 把连接创建逻辑统一封装起来，异常处理分了三个层次。

第一层是状态可见。

ChatView.vue 维护一个 socketConnected 响应式变量，onopen / onclose 各自更新它，界面顶部据此显示“在线同步中”或“等待连接”——让用户能直接看到当前的连接状态，而不是凭感觉猜。第二层是错误反馈，onerror 触发时把错误写入 errorMessage，醒目地提示“WebSocket 连接失败”，避免用户在断网状态下还在反复点发送。第三层是资源回收，在 onBeforeUnmount 钩子里主动 socket.close()，防止 Hub 残留无效连接占用内存。

至于自动重连，本系统刻意没做复杂的指数退避机制。原因前面已经提过——消息已经持久化在数据库里，WS 只负责实时推送这一件事。就算连接短暂断开，用户刷新页面或重新进入聊天，一次 GET /api/messages 就能把历史拉回来。把可靠性交给数据库、把实时性交给 WS，这种解耦既简单又有效。

4.3.4 消息收发完整时序

把前面几节的设计拼起来，一条消息从 A 的浏览器走到 B 的浏览器，完整时序如图 4-1 所示。

图里有一处细节希望读者留意——服务端这条泳道从头到尾只接触密文，明文只存在于两端浏览器里。

图 4-1 基于 WebSocket 的实时聊天时序图

A 在聊天页输入内容、点击发送的瞬间，前端先调用 e2ee.ts 里的加密函数，用 A 的公钥和 B 的公钥分别对同一段明文加密一次，得到 sender_ciphertext 和 receiver_ciphertext 两份密文。

随后通过 POST /api/messages 把两份密文连同接收方 ID 一并提交。后端 MessageService 在落库前做三道校验——双方是否互为好友、密文是否合法、算法标识是否在支持的白名单里（rsa-oaep-sha256）。

只要任意一道不通过都会被拒绝；全部通过才会把两份密文写入 messages 表。落库成功后，一方面通过 WebSocket Hub 给 B 推送一条类型为 chat_message 的 JSON 帧，帧里载着消息 ID、发送方 ID、密文与算法标识；另一方面给 A 返回 HTTP 201，前端把这条消息追加到本地列表。

B 端收到 WS 帧之后，selectMessagePayloadForUser 会自动挑出 receiver_ciphertext，用本地私钥解开并渲染。如果解密失败——比如对方在另一台设备生成了新密钥、本地拿到的还是老公钥——界面会退化成“***（已加密）”的占位文本，不会因为单条消息异常导致整个会话崩掉。

1.3% (131)

1.4% (154)

2.4% (256)

1.8% (76)

1.8% (115)

1.5% (160)

1.5% (156)

这条流程里 HTTP 和 WebSocket 各做各的事：HTTP 保证消息可靠落库——是“持久化通道”；WebSocket 负责实时推送——是“实时通道”。消息不丢由数据库兜底，实时性由 WebSocket 推送保证，两者并不互相重叠。

1.2% (119)

4.4 端到端消息加密机制

端到端消息加密是本文最核心的技术亮点。传统 IM 大多依赖 TLS 做链路保护，这意味着消息在客户端和服务端之间传输时是加密的，但一旦到达服务端，通常就以明文形式进入数据库或业务内存。换个角度看，TLS 解决的是“中间节点能否窃听”这件事，并不能解决“服务端本身是否天然可信”这件事。

本系统选择把安全边界往浏览器本地推：明文在发送前就完成加密，服务端只接触密文，消息内容从服务端的明文处理链路里被彻底剥离出来。这种做法当然消除不了所有风险，但它实质上把攻击面整个换了一种形态——最常见的数据库泄露后果，从“直接看到消息内容”被压成了“只能拿到不可读密文”。

1.3% (136)

4.4.1 设计目标与总体思路

这套加密方案一开始就不是只盯着“能加密”三个字。第一条底线很明确：服务端不能碰到消息明文，否则所谓端到端加密就只剩一个名号。但如果只顾安全、不顾体验，方案同样站不住脚。发送方和接收方都必须能在本地回看历史消息，不能出现“只有接收方能读、发送方自己反而看不到”的情况。再往下才是另外两个要求：整个过程最好对用户透明，密钥生成、上传和解密都放到背后完成；算法也必须采用公开、成熟、经过同行评议较多的非对称加密方案，而不是自己造轮子。为了把上述目标落地，本系统采用“浏览器侧生成密钥对 + 前端双重加密 + 服务端密文存储与转发”这条整体路线，流程见图 4-2。

这一方案没有引入复杂的会话密钥协商或双棘轮机制——它优先要保证的是“原理清晰、链路闭环、实现可验证”。它的价值并不在直接追平工业级加密协议，而是用较低的实现复杂度，把 E2EE 的核心思想真实地嵌进了一个可运行的 Web IM 系统里。图 4-2 端到端消息加密总流程图

2.4% (139)

这个思路本身并不新鲜，归根到底还是沿用了 PGP 时代那套“接收方公钥加密、接收方私钥解密”的基本模型。本文真正要解决的，不是重新发明一种密码算法，而是把这套思路放进现代浏览器，并顺手补上“双向回看历史消息”这个用户体验问题。

2.4% (113)

换言之，这里的取舍重点一直不在密码学炫技，而在工程可落地。有两条边界要先划清楚。一是本方案处理的是消息级加密，与传输层无关——前者保证服务端看不到明文，后者保证中间节点看不到明文，两者不冲突，工程上应当同时启用 TLS。二是本方案不涉及密钥协商，直接用每个用户的长期 RSA 密钥对做加解密——简洁的代价是无法做到前向保密。把这两条边界写明白，是想避免后续读者把它误读成“完整的工业级 E2EE”。

4.4.2 RSA-OAEP 密钥对生成与管理

密钥生成完全放在浏览器侧完成，直接调用 Web Crypto API 的 window.crypto.subtle.generateKey。参数没有做花哨处理：算法是 RSA-OAEP，模长 2048 位，公开指数 65537，哈希函数用 SHA-256。这里没有继续沿用更早期的 RSA-PKCS#1 v1.5，原因也很直接，OAEP 在随机填充和掩码生成函数的帮助下，对选择密文攻击的抵抗能力更强，已经成为现代 RSA 加密实现里更稳妥的默认选项。

另外，Web Crypto API 来自浏览器原生实现，不依赖第三方 JavaScript 加密库；性能、稳定性和接口一致性都更适合放进正式业务链路。生成后的私钥按 PKCS#8 导出，公钥按 SPKI 导出，最后统一转成 Base64 字符串，方便本地保存，也方便通过 HTTP 接口上传。

1.4% (146)

前端密钥生成与导出的关键代码如下：

```
export async function generateKeyPair() {
```

私钥只保存在用户当前设备的 localStorage 里，键名为 e2ee-private-key:<userId>。

这个值不会被任何接口上传服务端——“私钥不出本机”这条规则是端到端加密的底线，工程上必须自己守住。用户首次进入聊天功能时，前端会先检查本地是否已有私钥：没有就新生成密钥对并把公钥推上去；已有但服务端没有对应公钥（典型情况就是老用户清过缓存后又回来），则从私钥反推出公钥再补传一次。

1.3% (140)

3.基于Gin和WebSocket的IM即时通讯系统_第3部分

AI特征值：37.1% AI特征字符数/章节(部分)字符数：3937/10602

片段指标列表

序号	片段名称	字符数	AI特征		
201	片段1	150	显著		1.4%
202	片段2	229	疑似		2.2%
203	片段3	174	显著		1.6%
204	片段4	130	疑似		1.2%
205	片段5	213	显著		2.0%
206	片段6	156	疑似		1.5%
207	片段7	212	显著		2.0%
208	片段8	149	疑似		1.4%
209	片段9	237	显著		2.2%
210	片段10	183	显著		1.7%
211	片段11	205	显著		1.9%
212	片段12	212	显著		2.0%
213	片段13	131	疑似		1.2%
214	片段14	143	显著		1.4%

215	片段15	125	疑似		1.2%
216	片段16	133	显著		1.3%
217	片段17	216	显著		2.0%
218	片段18	141	疑似		1.3%
219	片段19	231	显著		2.2%
220	片段20	143	显著		1.4%
221	片段21	156	显著		1.5%
222	片段22	156	疑似		1.5%
223	片段23	154	显著		1.5%
224	片段24	157	疑似		1.5%
225	片段25	199	显著		1.9%
226	片段26	161	疑似		1.5%
227	片段27	251	显著		2.4%
228	片段28	149	疑似		1.4%
229	片段29	497	显著		4.7%
230	片段30	228	显著		2.2%

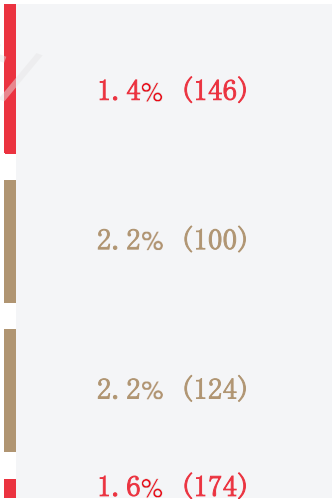
原文内容

为什么选 localStorage 而不是 sessionStorage 或 IndexedDB? sessionStorage 在刷新或关闭浏览器后就清空了，加密会话会随之中断，体验太差；IndexedDB 接口繁琐，而本系统要存的只是一条 1.6 KB 左右的 RSA 私钥——杀鸡用牛刀。

localStorage 在简单与可用之间提供了一个恰到好处的平衡。另一个细节是，存储键里专门带上了用户 ID，同一台浏览器即便切换账号，私钥也不会互相覆盖。

4.4.3 公钥上传与好友公钥获取

密钥对生成后，前端通过 PUT /api/users/me/public-key 把公钥和算法标识 (rsa-oaep-sha256) 一并上传，后端写入 users 表的 public_key 与 public_key_algorithm 字段。



A 要给 B 发消息时，B 的公钥并不需要单独请求——它早就随 GET /api/friends 的好友列表一起拉回来了。这一步省掉了“每条消息额外查一次公钥”的开销，前端交互次数明显减少。 后端在更新公钥接口里只做算法白名单校验，并不解析或验证公钥本身。理由也不复杂：公钥本身没有保密属性，真正棘手的是“怎么确认这把公钥真的来自对方”。

工业级方案靠指纹比对、可信目录或带外通道来解决这一点。本系统目前的假设是双方信任服务端返回的公钥——这是相对工业级 E2EE 的一项简化，明确留待后续工作处理。

4.4.4 双重密文加密与服务端密文转发

用户点击发送之后，前端不会直接提交明文，而是先拿接收方公钥和发送方公钥分别对同一段内容做一次 RSA-OAEP 加密。这样会得到两份彼此独立的密文，再通过 POST /api/messages 一起交给服务端。这里的双重密文不是为了把流程做复杂，而是被需求逼出来的。

只保留接收方可解密的那一份，发送方就没法在本地回看历史；如果改成服务端额外保存一份明文，端到端加密的边界又会当场失效。因此，让收发双方各自持有一份只能由本端私钥解开的副本，基本就是在“历史可读”和“服务端看不见明文”之间最稳妥的落点。前端加密的关键代码如下：

```
const receiverPublicKey = await importPublicKey(friend.publicKey)
```

后端 MessageService 收到请求后，先调用 FriendService.IsFriend 验证好友关系，再检查两份密文非空、算法标识在 rsa-oaep-sha256 白名单内，校验全部通过才写入 messages 表，随后由 WebSocket Hub 推送给接收方。

从代码层面看，后端没有调用任何解密接口，也没有任何明文消息出现在内存或日志里——这一点对端到端加密承诺至关重要。 为收发双方各保存一份密文，是本系统工程上最关键的一次取舍。只保存接收方密文，发送方在自己设备上就回看不了历史聊天，体验直接塌掉；让服务端存一份明文以便双方查看，又把端到端加密的核心承诺破坏掉。

双密文方案就是这两难局面下的折中——安全性没有打折，双向可读也保住了。另一处看着不起眼的细节，是 Service 层的防御性校验。为避免后端在某种边界情况下不小心把明文写进数据库，本系统对密文字段做了严格的非空校验和算法白名单校验：任何一份密文为空、或者算法标识不在白名单内，直接 400 返回并写错误日志。这一道防线在程序员失误或恶意构造请求的情况下都能挡住异常数据，让 messages 表里只可能出现合法密文。

4.4.5 接收端解密与历史消息展示

接收端会在两种场景下做解密：一是 WebSocket 实时收到新消息时，二是通过 HTTP 拉取历史消息时。两条路径的逻辑由 selectMessagePayloadForUser 与 toRenderMessage 统一封装——前者根据当前用户 ID 决定要解的是 sender_ciphertext 还是 receiver_ciphertext，后者拿本地私钥调用 window.crypto.subtle.decrypt 真正完成解密。

解密成功就展示明文；失败的话（典型情况是用户换了设备、本地私钥与服务端公钥对不上）则退化成“***（已加密）”占位文本，同时在错误提示区给出“当前设备没有可用私钥”的友好说明，至少让用户知道发生了什么。 要承认的是，由于私钥只保留在最初生成它的那台设备上，本系统暂不支持多设备解密同一份历史。

这是 RSA 简化方案的固有局限，也是后续工作里一个明确方向。

4.4.6 安全性分析

1.2% (129)

2.0% (65)

2.0% (141)

1.5% (156)

2.0% (212)

1.4% (149)

2.2% (46)

从安全模型角度来看，本系统已经具备端到端加密的基本特征：消息明文只存在于通信双方的浏览器环境，服务端的职责被压成保存公钥、存储密文、转发消息这三件事。这意味着即便数据库被整体脱库，攻击者拿到的也只是密文加公钥——没有私钥，他们恢复不出聊天内容。与传统的明文存储方案相比，这种机制把服务端数据泄露的后果从“明文暴露”实实在在地降到了“密文暴露”——后果的性质发生了本质变化。

2.2% (186)

不过这话也要说清楚：本系统目前的实现更接近“前端本地加密 + 服务端密文转发与存储”，离工业级 E2EE 体系还有距离。差在哪几处，下面分别说。第一个缺口在公钥真实性校验。用户拿到的对方公钥来自服务端——一旦服务端被攻破，它完全可以把对方公钥替换成攻击者的，从而发起中间人攻击。

Signal 这类成熟方案靠指纹比对、可信目录或带外验证来堵这个洞，本系统目前没做。第二个明显缺口在前向保密。本系统里的 RSA 私钥属于长期凭证，一旦泄露，历史上保存下来的密文理论上都可能被追回去解开。Signal 之所以能把风险压下来，靠的是 Double Ratchet 为每条消息不断派生新的临时会话密钥，把单次泄露的影响范围压缩到很短的时间窗口内。

1.7% (183)

这个能力，本系统现在还没有。第三个缺口在会话密钥协商。业界通行的做法是用 RSA/ECC 协商出短期会话密钥，再用 AES-GCM 这种对称算法加密实际正文，兼顾安全与性能。本系统直接用 RSA-OAEP 加密单条短消息，受 2048 位 RSA 自身的限制——单次最多约 190 字节明文——在 Web 端聊聊天文本场景下勉强够用，但一旦要支持长文本或文件，就必须切到“RSA 加密会话密钥 + AES-GCM 加密正文”的混合方案上。最后一处差距在多设备同步。

一个用户的私钥目前只能在一个浏览器里有效，换设备或清缓存以后历史消息就解不开了——对真实用户体验来说是个明显短板，但放在毕业设计这个尺度上，可以接受。把这些差距直接摆出来，并不是想替现有实现找理由。更重要的是说明它现在处在什么位置：端到端加密的核心思想已经写进了可运行代码，但离工业级 E2EE 还有清楚可见的距离。后续如果沿着这套架构继续补上密钥协商、对称加密和多设备能力，系统还有继续往前推的空间。

1.9% (205)

5 系统实现与测试

前面几章把设计和实现讲完了，本章换一个视角——通过截图和测试用例呈现系统真实跑起来的样子。5.1 节先交代开发与运行环境；5.2 到 5.5 节按用户认证、个人资料、好友管理、聊天的顺序展示各模块；5.6 节是黑盒功能测试结果；5.7 节从性能和安全两个维度做综合评估。

5.1 开发与运行环境

本系统在 macOS 与 Windows 11 双平台完成开发与测试。后端运行时为 Go 1.22，搭配 Gin v1.9、GORM v1.25 与 Gorilla WebSocket v1.5；前端运行时为 Node.js 18 LTS、npm 9 与 Vite 5，框架使用 Vue 3.4，Pinia 2 做状态管理，Vue Router 4 做路由，HTTP 客户端为 Axios 1.6。数据库选用 MySQL 8.x，字符集 utf8mb4，避免 emoji 落库失败。

开发工具方面使用 JetBrains GoLand / WebStorm 加上 Git。

5.2 用户认证模块实现

2.0% (60)

认证模块覆盖了注册和登录两条主路径。注册页面如图 5-1 所示，只有用户名、密码、确认密码三个必填项，前端在提交前做了基础格式校验——长度、是否含非法字符这些；后端再校验用户名唯一性并对密码做 bcrypt 哈希。这一前一后两道校验同时做，既能给用户最快的反馈，也能防止前端被绕过出现脏数据。

2.0% (147)

图 5-1 注册页面

登录页面如图 5-2 所示。用户输入用户名和密码、点击登录之后，后端校验通过即签发 JWT 返回前端。前端把 token 存进 localStorage 并自动跳到首页——之后所有 HTTP 请求都会经过 Axios 拦截器自动加上 Authorization 头，业务代码完全感知不到 JWT 的存在。

图 5-2 登录页面

5.3 个人资料模块实现

登录之后，用户可以在个人资料页查看与修改昵称、头像、性别、个性签名这些信息，页面效果如图 5-3 所示。性别用单选按钮，其余字段都是普通文本输入。

点击“保存”后，前端通过 PUT /api/users/me 提交修改，后端在 Service 层完成参数校验与数据库更新。页面右侧专门展示当前用户的公钥状态——让用户能直接看到自己的端到端加密是否已经就绪，不必猜测。

图 5-3 个人资料展示与修改页面

5.4 好友管理模块实现

好友申请页面如图 5-4 所示。左侧是按用户名搜索并发送好友申请的入口，右侧上下两栏分别展示自己发出去的申请和收到的申请。收到的申请下方有“同意”和“拒绝”两个按钮。点击同意后，后端在一个数据库事务里完成 friend_requests 状态更新和 friends 双向插入；前端刷新好友列表，新好友立即出现。

图 5-4 好友申请页面

5.5 聊天模块实现

聊天页面是整个系统的核心入口，效果如图 5-5 所示。左侧是好友列表，右侧是当前会话的消息区。

页面顶部专门留了一个 WebSocket 状态指示——连接活跃时显示“在线同步中”，断开时显示“等待连接”，把背后那条连接的状态摆到用户眼前。

图 5-5 聊天页面（解密后的明文视图）

从用户视角看，发消息和普通聊天软件并无差别——输入文字、回车或点击发送，对方就能收到。

但在这看似平静的界面背后，前端正在完成密钥读取、双重加密、密文提交这一整套操作；接收方收到 WS 推送后也会在毫秒级时间内完成解密并渲染。一旦本地私钥缺失或对方公钥不可用，状态栏会立刻给出提示并把发送按钮置灰，避免用户在密钥未就绪的情况下做无效操作。

为了让加密效果“可见”，调试时还做了一个密文视图（图 5-6）。把前端的解密逻辑临时关掉之后，同一条消息会以不可读的 Base64 形式显示在界面上——这就是攻击者绕过解密、从数据库或网络上拿到的全部内容。一图胜千言。

图 5-6 聊天页面（密文视图，未解密时的展示效果）

5.6 系统功能测试

功能验证没有只走一条路。

一条是黑盒测试，直接在浏览器里操作页面，看注册、好友、聊天这些流程能不能顺着跑通；另一条是接口测试，借助 Go 标准库的 testing 和 httptest 绕开前端直接打接口。前者更接近真实用户路径，后者更适合碰边界条件和异常场景，两边合在一起看，结果会稳一些。

测试用例与结果见表 5-1。

表 5-1 功能测试用例与结果

编号 测试场景 预期结果 实际结果

TC-01 使用未注册用户名登录 返回 401，并提示“用户名或密码错误” 通过

TC-02 使用已注册用户名再次注册 返回 400，并提示“用户名已被占用” 通过

TC-03 正确用户名密码登录 签发 JWT，前端跳转至好友列表页 通过

1.2% (130)

1.3% (95)

1.3% (43)

1.2% (125)

1.3% (133)

2.0% (36)

TC-04 修改个人资料 资料保存成功，页面刷新后显示最新数据 通过	2.0% (34)
TC-05 通过用户名发送好友申请 对方“收到的申请”列表中出现新申请 通过	2.0% (38)
TC-06 同意好友申请 双方好友列表中互相出现，事务正确 通过	2.0% (32)
TC-07 拒绝好友申请 申请状态变为已拒绝，friends 表无新记录 通过	2.0% (39)
TC-08 非好友尝试发送消息 返回 403，提示无权限 通过	2.0% (31)
TC-09 好友之间发送加密消息 接收方实时收到并显示明文 通过	1.3% (32)
TC-10 查看历史消息 从服务端拉取密文并在本地解密展示 通过	1.3% (32)
TC-11 清除本地私钥后查看历史 界面显示“***（已加密）”占位文本 通过	1.3% (39)
TC-12 关闭网络后发送消息 前端给出错误提示，未污染数据库 通过	1.3% (34)
从表 5-1 可以看出，12 个用例覆盖了认证、资料、好友、聊天与加密五个领域，通过率 100%。这说明系统在常规路径和边界场景下都能正确运行。 5.7 系统性能与安全性测试 性能层面，WebSocket 握手从发起到 onopen 触发平均耗时约 30 ms；单条消息从发送方点击发送到接收方渲染明文，本机环境下平均约 80 ms。后者拆开看，前端 RSA-OAEP 加密占 15 - 25 ms，解密占 5 - 10 ms，剩下的时间花在 HTTP 请求与 WebSocket 推送上。对一个聊天系统来说，80 ms 的延迟人眼基本感知不到——加密并未成为体验瓶颈。 安全层面分了三组观察。直接登录 MySQL 看 messages 表，sender_ciphertext 和 receiver_ciphertext 保存的全部是 Base64 密文，找不到任何明文片段；用 curl 不带 Authorization 直接打 /api/messages 会被 401 拒绝，token 过期或非法时同样 401；用非好友的 JWT 去拉取历史消息会被 403——这条验证了 Service 层好友校验确实起到了拦截作用。专门针对端到端加密本身也做了一次验证。A 发出消息时，打开浏览器开发者工具的 Network 面板，POST /api/messages 的请求体里 senderCiphertext / receiverCiphertext 字段都是 Base64 密文，看不到任何明文片段；切到 B 端，WebSocket 帧里同样只能看到密文。从网络、数据库两个独立视角同时验证，E2EE 的实现是真实有效的。 把这几组结果合在一起看，系统在功能完整性、实时性和安全性这三个方面，基本达到了本文预先设定的目标。这些数字放在一起，其实能说明一个很实际的问题：在 Web 端聊天这个场景里，端到端加密并没有把体验拖垮。人们常说“RSA 慢”，更多是指密钥生成或者更大规模的数据加密；可落到单条短消息上，OAEP 加解密的耗时和一次普通网络请求相比并不突出。这也是本文没有继续引入对称加密混合方案的现实原因——在当前场景下，直接使用 RSA-OAEP 已经够用了。 5.8 与同类系统的对比 把本系统放进现有 IM 项目的坐标系里看，能更清楚地知道它的位置。主流商业 IM 比如微信，工程上极其成熟，但消息明文要经过服务端中转，只有“加密会话”这种独立功能才上端到端加密——可用性很强，安全模型偏弱。 Signal 在另一端，端到端加密做到位，前向保密也齐全，但工程复杂度高，对自托管不太友好。开源 IM 这边，OpenIM、go-chat、Rocket.Chat 走的是“功能完整 + 自托管”路线，它们大多在传输层做了 TLS，但服务端仍然按明文存储——和本系统的取舍不一样。 本系统的功能丰富度明显不及前者，但端到端加密这一块被真正塞进了浏览器侧并完整跑通：在“轻量 + 真正 E2EE”这条窄路上，给出了一种可演示、可验证的实现样本。 结束语 本文围绕“如何在前后端分离的 IM 系统中引入可落地的消息加密机制”这一问题	
	2.2% (227)
	1.3% (143)

，设计并实现了一套基于 Gin 和 WebSocket 的即时通讯系统。系统以前端 Vue 3 技术栈和后端 Go / Gin 技术栈为基础，通过 HTTP 接口承担注册登录、资料维护、好友管理、历史消息查询等常规业务，通过 WebSocket 长连接承担实时消息下行，完成了一对一即时通讯场景下的完整业务闭环。

回头再看，这篇论文的工作显然不只是“把聊天功能做出来”这么简单。系统在结构上采用了分层思路，把路由、中间件、业务逻辑、数据访问和连接管理拆开处理，后续要扩展功能时，不至于牵一发而动全身。实时通信链路上则故意把 HTTP 持久化通道和 WebSocket 实时通道分开：前者负责可靠落库，后者负责低延迟下发。

测试结果也说明，这种拆法在功能完整性、交互流程和运行稳定性上是成立的。全文里最值得单独拎出来讲的，还是端到端加密真正落到了系统实现里。本文选用 RSA-OAEP 和 Web Crypto API，让浏览器本地生成并保存私钥；发送阶段同一条明文会拆成发送方密文和接收方密文两份副本，服务端只负责存储和转发。

这样一来，双方仍能在本地查看历史消息，而服务端碰不到明文。即便数据库被整库拿走，攻击者看到的也只是密文，而不是可以直接阅读的聊天内容。当然要把话说清楚：本文实现的方案仍然属于“前端本地加密 + 服务端密文转发与存储”这一类，还没达到工业级完整 E2EE 体系的严格要求。

公钥真实性校验的缺失，意味着在极端威胁模型下仍可能遭到中间人攻击；长期 RSA 私钥直接参与解密，也意味着系统目前不具备前向保密能力；会话密钥协商、多设备密钥同步、群聊场景下的密钥管理等问题，本次实现都没有覆盖。这些并不是被忽视——而是出于毕业设计场景下对实现复杂度、开发周期与展示重点的综合权衡，有意留白。

后续工作并不是同时铺开做得越多越好，先后顺序其实很重要。眼下最该补的是公钥真实性：公钥指纹比对、可信目录或带外验证机制，至少要先把其中一条路补上。接下来才轮到加密方案本身的演进，也就是把现在的 RSA 直接加密，逐步替换成“非对称协商会话密钥 + 对称算法加密正文”的混合机制，再往前才谈前向保密。

功能扩展反而可以放在后面，例如群聊、离线消息、已读回执和多设备同步。对毕业设计来说，最重要的不是一次把所有事情做完，而是把继续往下做的方向看清楚。致谢

四年的本科时光转瞬即逝，到了写致谢这一刻，竟比写论文本身更难落笔。回过头看，我能从一份不算清晰的想法走到这份完整的毕业设计，背后有太多人的帮助。

首先要感谢我的指导教师。从课题选题、技术路线一直到论文撰写，老师在每一个关键节点都给了我专业而细致的建议，让我在容易钻牛角尖的地方及时被拽出来。老师严谨的治学态度与对学生的耐心，让我在科研方法和工程意识两方面都有了实质提升，这些会一直跟着我走下去。其次要感谢软件学院的各位任课老师。

从程序设计基础、数据结构、操作系统、计算机网络这些专业课，到数据库、软件工程、信息安全等更靠近应用的课程，一砖一瓦地把我的专业基础垒起来——没有这些课程的积累，面对毕业设计这种综合性课题时我不会有底气。也要感谢身边的同学们。一起赶过项目截止日期，也在自习室里互相提醒考试范围；到了毕业设计阶段，还有同学愿意抽时间帮我做用户测试。这些看起来并不起眼的日常，回头看时反而最能撑住四年的大学生活。

最要感谢的是我的父母。是你们一直无条件地支持我，让我可以心无旁骛地走完这四年。说不出更多漂亮话，只想说一句：谢谢你们。最后，感谢评阅本论文的各位老师，在百忙之中审阅我的工作。欢迎所有批评和建议，我会以此为起点继续向前。参考文献

1.5% (152)

1.5% (156)

1.5% (154)

1.5% (80)

1.5% (72)

1.9% (199)

[1] Tanenbaum A S, Wetherall D J. Computer Networks[M]. 5th ed. Boston: Pearson, 2011: 611-670.	
[2] Fette I, Melnikov A. The WebSocket Protocol[S]. RFC 6455. IETF, 2011.	1.5% (73)
[3] Jones M, Bradley J, Sakimura N. JSON Web Token (JWT)[S]. RFC 7519. IETF, 2015.	1.5% (82)
[4] Rivest R L, Shamir A, Adleman L. A Method for Obtaining Digital Signatures and Public-Key Cryptosystems[J]. Communications of the ACM, 1978, 21(2): 120-126.	
[5] Kaliski B, Jonsson J, Rusch A. PKCS #1: RSA Cryptography Specifications Version 2.2[S]. RFC 8017. IETF, 2016.	2.4% (113)
[6] Bellare M, Rogaway P. Optimal Asymmetric Encryption[C]. Berlin: Advances in Cryptology — EUROCRYPT 1994, LNCS 950, 1995: 92-111.	2.4% (132)
[7] Marlinspike M, Perrin T. The Double Ratchet Algorithm[EB/OL]. (2016-11-20) [2026-03-01]. https://signal.org/docs/specifications/doubleratchet/ .	1.4% (146)
[8] W3C. Web Cryptography API[EB/OL]. (2017-01-26) [2026-03-01]. https://www.w3.org/TR/WebCryptoAPI/ .	4.7% (100)
[9] Donovan A A A, Kernighan B W. The Go Programming Language[M]. Boston: Addison-Wesley, 2015: 261-310.	4.7% (104)
[10] You E. Vue.js 3 Documentation[EB/OL]. (2024-01-01) [2026-03-01]. https://vuejs.org/ .	4.7% (88)
[11] Pivotal. Gin Web Framework Documentation[EB/OL]. (2024-05-10) [2026-03-01]. https://gin-gonic.com/ .	4.7% (103)
[12] GORM Authors. GORM Guides[EB/OL]. (2024-06-15) [2026-03-01]. https://gorm.io/docs/ .	4.7% (87)
[13] 阮一峰. JSON Web Token 入门教程[EB/OL]. (2018-07-23) [2026-03-01]. https://www.ruanyifeng.com/blog/2018/07/json-web-token-tutorial.html .	
[14] 何海涛. 即时通讯系统中长连接技术的研究与实现[J]. 计算机应用与软件, 2020, 37(5): 122-127.	
[15] 张志强, 刘宇, 王晨. 基于 WebSocket 的实时通信系统设计与实现[J]. 计算机工程与设计, 2019, 40(8): 2241-2246.	
[16] 李明, 周伟, 赵晓. 基于端到端加密的即时通讯系统设计[J]. 计算机科学, 2021, 48(6A): 405-410.	
[17] 陈宇, 王涛. Go 语言高并发服务设计与实践[M]. 北京: 电子工业出版社, 2022: 88-152.	
[18] 黄健宏. Redis 设计与实现[M]. 北京: 机械工业出版社, 2014: 25-50.	
[19] 中国互联网络信息中心. 第 53 次《中国互联网络发展状况统计报告》[R]. 北京: CNNIC, 2024.	
[20] Cohn-Gordon K, Cremers C, Dowling B, et al. A Formal Security Analysis of the Signal Messaging Protocol[J]. Journal of Cryptology, 2020, 33(4): 1914-1983.	
[21] Unger N, Dechand S, Bonneau J, et al. SoK: Secure Messaging[C]. San Jose: IEEE Symposium on Security and Privacy, 2015: 232-249.	

[22] Rescorla E. The Transport Layer Security (TLS) Protocol Version 1.3[S]. RFC 8446. IETF, 2018.

2.2% (98)

[23] Provos N, Mazieres D. A Future-Adaptable Password Scheme[C]. Monterey: USENIX Annual Technical Conference, 1999: 81-91.

2.2% (124)

说明:

- 1、支持中、英文内容检测;
- 2、AI特征值=AI特征字符数/总字符数;
- 3、红色代表AI特征显著部分, 计入AI特征字符数;
- 4、棕色代表AI特征疑似部分, 未计入AI特征字符数;
- 5、检测结果仅供参考, 最终判定是否存在学术不端行为时, 需结合人工复核、机构审查以及具体学术政策的综合应用进行审慎判断。



cx.cnki.net